

PROGETTO SDCC B1

Distributed Service Registry con Replica Gossip

Corso di Sistemi Distribuiti e Cloud Computing (SDCC)

I. INTRODUZIONE E OBIETTIVI

In un sistema composto da microservizi, le istanze applicative possono nascere, terminare o cambiare collocazione durante l'esecuzione. Un catalogo statico di indirizzi non riesce quindi a rappresentare in modo affidabile lo stato corrente dell'infrastruttura. Serve un componente capace di raccogliere le registrazioni, verificarne la vitalità e restituire ai client un endpoint aggiornato.

Questo progetto sviluppa in Go un **registro di servizi distribuito**. Il registro conserva nome, identificativo, endpoint, versione e stato di salute delle istanze applicative. La sua caratteristica centrale è l'assenza di un punto di controllo unico: tre nodi registry condividono le informazioni attraverso gossip e sincronizzazioni periodiche.

Gli obiettivi sono dimostrare la discovery decentralizzata, la convergenza delle viste locali, la continuità del servizio dopo il guasto di un nodo e il recupero automatico dello stato. L'intero ciclo è riproducibile tramite Docker Compose e lo script operativo `scripts/dev.ps1`.

II. ARCHITETTURA DEL SISTEMA

Il sistema adotta un modello peer-to-peer nel quale ogni registry mantiene una copia locale del catalogo e della membership. I nodi non eleggono un coordinatore permanente: ciascuno può ricevere richieste esterne, propagare aggiornamenti e recuperare informazioni dagli altri peer.

I requisiti principali sono:

- **Disponibilità e tolleranza ai guasti:** la discovery deve continuare a rispondere anche quando una parte del cluster non è raggiungibile, evitando una dipendenza da un singolo server.
- **Consistenza eventuale senza leader:** le modifiche devono raggiungere progressivamente tutti i nodi mediante gossip, senza introdurre un coordinamento centrale o un consenso sincrono.
- **Recovery post-crash:** un peer riavviato deve poter ricostruire membership e dati persi sfruttando seed peer, gossip e reconcile.
- **Operabilità e riproducibilità:** build, test, deployment e fault injection devono essere eseguibili tramite Docker Compose e un runner unico.

L'architettura espone due interfacce gRPC. `ServiceRegistry` gestisce il rapporto tra servizi applicativi e catalogo; `RegistryPeer` gestisce membership e replica tra registry. I messaggi sono definiti con Protocol Buffers e trasformati in codice Go tipizzato,

riducendo ambiguità nel contratto di rete e il costo della serializzazione.

III. CONTESTO, REQUISITI E SCENARIO DI ESECUZIONE

A. Contesto architetturale e modello di deployment

Il sistema è pensato per un ambiente distribuito e parzialmente isolato, rappresentato localmente tramite **Docker**. Docker Compose crea una rete bridge condivisa nella quale i nodi possono risolversi attraverso i rispettivi nomi DNS, ad esempio `registry-node-1`, `registry-node-2` e `registry-node-3`.

Ogni registry vive in un container separato e mantiene una **vista locale** composta da catalogo dei servizi e membership dei peer. I tre nodi utilizzano lo stesso binario, ma ricevono configurazioni differenti per identificativo, indirizzo pubblicizzato e volume di persistenza.

B. Requisiti operativi e prerequisiti di sistema

Per rendere l'esecuzione ripetibile, l'ambiente host deve disporre dei seguenti strumenti:

- **Docker Desktop:** gestisce immagini, container, rete virtuale, volumi e mapping delle porte gRPC.
- **PowerShell:** esegue `scripts/dev.ps1`, che raccoglie i comandi di progetto e automatizza lo scenario.
- **Go 1.24 o successivo:** permette di compilare il codice e lanciare la suite di test sull'host.

C. Scenario dimostrativo ed evoluzione temporale

La validazione funzionale è organizzata come una sequenza di comandi forniti da `scripts/dev.ps1`. Il flusso combina `trace-up`, `select-service`, `register`, `discovery`, `deregister` e `down` per riprodurre il ciclo di vita di un servizio all'interno del cluster.

Lo scenario copre le seguenti macro-fasi sequenziali:

- 1) **Bootstrap del cluster:** `trace-up` ricrea l'ambiente, prepara le immagini e avvia i tre registry.
- 2) **Scelta e registrazione:** `select-service` seleziona il profilo e `register` invia il record attraverso la CLI containerizzata, avviando anche il heartbeat.
- 3) **Convergenza:** il gossip diffonde il record e i peer verificano progressivamente lo stesso stato.
- 4) **Monitoraggio:** heartbeat e TTL permettono di distinguere un servizio attivo da uno non più raggiungibile.
- 5) **Fault injection:** l'arresto di un registry simula un guasto parziale.

- 6) **Continuità operativa:** i peer superstiti rispondono alle richieste e mantengono la copia replicata.
- 7) **Recovery:** il nodo viene riavviato e recupera lo stato mediante gossip e *reconcile*.
- 8) **Teardown: deregister** propaga la rimozione logica e **down** arresta l'ambiente.

IV. INTERFACCE GRPC E MODELLO DATI

I contratti di comunicazione sono descritti con **Protocol Buffers (v3)** e compilati in stub Go. Il file `proto/registry.proto` separa le operazioni rivolte ai servizi applicativi da quelle necessarie alla cooperazione tra registry.

1) Definizione dei Servizi gRPC:

- **ServiceRegistry (interfaccia client-to-cluster):** governa il ciclo di vita delle istanze che utilizzano la discovery. Espone le seguenti procedure remote:
 - **RegisterService:** inserisce un'istanza con i relativi dati di rete e metadati.
 - **DeregisterService:** avvia la rimozione controllata di un'istanza.
 - **Heartbeat:** rinnova la presenza e aggiorna lo stato di salute.
 - **GetService / ListServices:** permettono di risolvere una singola istanza o consultare il catalogo locale.
- **RegistryPeer (interfaccia peer-to-peer):** definisce le RPC con cui i nodi condividono membership e dati:
 - **JoinCluster:** annuncia un nodo ai seed peer e recupera la membership conosciuta.
 - **GossipSync:** invia aggiornamenti locali a un insieme limitato di peer.
 - **PullState:** recupera lo stato di un peer durante la riconciliazione o il recovery.

2) *Modello Dati e Struttura del ServiceRecord:* Il record principale scambiato tra client e peer è **ServiceRecord**. Oltre ai dati del servizio, contiene informazioni necessarie per controllare vitalità, proprietà e ordine degli aggiornamenti.

I campi principali che compongono il messaggio sono così strutturati:

- **Identificazione univoca:**
 - **service_name** (string): nome logico usato per raggruppare istanze dello stesso servizio.
 - **service_id** (string): identificativo della singola istanza, distinto dal nome del servizio.
- **Metadati applicativi e di rete:**
 - **endpoint** (string): indirizzo host e porta al quale il client può collegarsi.
 - **version** (string): versione dell'istanza, utile per distinguere release differenti.
- **Stato di salute e vitalità:**

- **health_status** (enum): stato osservato dell'istanza, ad esempio `serving` o `degraded`.
- **last_heartbeat_unix** (int64): timestamp Unix dell'ultimo keep-alive ricevuto, usato per il calcolo del TTL.

• Proprietà distribuite e ordinamento:

- **owner_node_id** (string): nodo che ha accettato la registrazione o riceve gli heartbeat diretti.
- **logical_version** (uint64): contatore monotono usato per accettare soltanto aggiornamenti più recenti e ignorare duplicati o messaggi fuori ordine.

V. ARCHITETTURA, ORCHESTRAZIONE E SINCRONIZZAZIONE DEL CLUSTER

Il sistema segue un approccio *Docker-first*: Docker Compose gestisce i container e `scripts/dev.ps1` offre un'interfaccia unica per le operazioni di sviluppo. Il ciclo operativo distingue il reset iniziale, l'avvio dei nodi e la successiva convergenza delle loro viste.

A. Avvio ed Orchestrazione del Cluster

Il comando `scripts/dev.ps1 trace-up` esegue una sequenza deterministica di preparazione dell'ambiente:

Le operazioni principali sono:

- **Reset dello stato:** `docker compose down -v` elimina container, rete e volumi della sessione precedente.
- **Build:** vengono costruite le immagini dei tre registry e del client CLI.
- **Preparazione:** viene creato il file locale che segnala l'avvio dello scenario.
- **Avvio:** i tre nodi registry vengono avviati; il heartbeat parte in seguito con **register**.

La chiusura ordinaria si esegue con `scripts/dev.ps1 down`: i container vengono rimossi, mentre i volumi Docker non vengono cancellati.

B. Topologia di Docker Compose e Modello di Rete

Il file `deploy/docker-compose.yml` descrive quattro servizi logici collegati da una rete virtuale *bridge* ($\mathcal{N}_{\text{bridge}}$). Siano $N = \{n_1, n_2, n_3\}$ i registry e C_{cli} il client effimero. I registry condividono l'immagine applicativa e si distinguono attraverso il profilo YAML assegnato.

La mappatura delle porte tra l'interfaccia di loopback dell'host (P_{host}) e le porte interne ai container ($P_{\text{container}}$) risponde alla seguente funzione di partizionamento:

$$f(n_i) : P_{\text{container}} \rightarrow P_{\text{host}}(n_i) \quad \text{dove} \quad P_{\text{container}} = 50051 \quad (1)$$

$$\begin{cases} f(n_1) \Rightarrow 50051 \rightarrow 50051 \\ f(n_2) \Rightarrow 50051 \rightarrow 50052 \\ f(n_3) \Rightarrow 50051 \rightarrow 50053 \end{cases} \quad (2)$$

La rete consente ai container di risolvere i peer tramite hostname Docker, per esempio **registry-node-1**. Il container **service-cli** non è un demone: viene creato solo quando serve e rimosso al termine del comando.

C. Bootstrapping Interno e Ciclo di Vita del Processo Go

Quando il binario Go viene avviato, il processo segue quattro passaggi principali:

- 1) **Lettura della configurazione:** il flag `-config` seleziona il profilo YAML del nodo.
- 2) **Rete e persistenza:** viene aperto il listener e, se presente, viene caricato lo snapshot dallo `storage_dir`.
- 3) **Server gRPC:** vengono registrate le API per client e peer sullo stesso server.
- 4) **Runtime asincroni:** vengono avviati gossip, reconcile, membership sweep e ticker per i servizi stale.

Alla ricezione di SIGINT o SIGTERM il processo ferma i runtime, comunica il proprio *Graceful Leave* ai peer e arresta il server gRPC in modo ordinato.

D. Topologia Gossip e il Meccanismo del Gossip Gate

I profili YAML regolano gossip e membership attraverso parametri come `listen_address`, usato per il binding interno, e `advertise_address`, usato dai peer per raggiungere il nodo.

Per evitare che i nodi inizino a scambiarsi dati prima che il cluster sia pronto, il runtime usa una barriera logica chiamata **Gossip Gate**. Sia G_i lo stato del gate del nodo i ed E l'esistenza del file di controllo:

$$G_i = \begin{cases} \text{BLOCKED}, & \neg \exists(\text{gossip_gate_path}) \\ \text{UNBLOCKED}, & \exists(\text{gossip_gate_path}) \end{cases} \quad (3)$$

I loop di bootstrap, gossip, reconcile e peer sweep attendono l'apertura del gate:

$$\forall \text{loop} \in \{\text{Gossip, Reconcile, Sweep}\}, \quad \text{Attendi}(G_i == \text{UNBLOCKED}) \quad (4)$$

La transizione da BLOCKED a UNBLOCKED avviene durante `scripts/dev.ps1 register`, che crea `.gossip-enabled` nei tre container.

E. Esecuzione dei Comandi tramite CLI Tool Effimera

Il container `service-cli` esegue comandi brevi nella rete Docker tramite `docker compose run -rm service-cli [subcommand]`. Le operazioni disponibili sono:

$$\text{Subcommands} = \{\text{register, heartbeat, deregister, get, list}\} \quad (5)$$

Se uno o più peer sono spenti, il client riconosce gli errori di disponibilit  e segnala il nodo disattivato invece di interrompere il processo con un'eccezione non gestita. Il runner PowerShell permette inoltre di distinguere la build iniziale dai comandi CLI successivi, evitando ricostruzioni inutili.

F. Gestione dello stato dei servizi e mutazioni di stato

Il modulo **ServiceStore** rappresenta in memoria lo stato locale delle istanze. Ogni modifica passa da normalizzazione, validazione e versionamento, cos  da rendere il comportamento prevedibile anche quando gli aggiornamenti arrivano in modo asincrono.

1) *Meccanismo di Rinnovo (Heartbeat):* Il servizio mantiene la propria presenza inviando keep-alive. Alla RPC **Heartbeat**, il nodo controlla che l'istanza esista e non sia una tombstone, quindi aggiorna timestamp e salute. Un heartbeat riferito a un servizio sconosciuto o gi  rimosso viene rifiutato e richiede una nuova registrazione.

2) *Cancellazione Logica tramite Tombstone:* Per evitare che un vecchio messaggio faccia riapparire un servizio rimosso, il progetto usa il pattern **Tombstone**:

- **DeregisterService** svuota endpoint e versione applicativa.
- Lo stato viene portato a **NOT_SERVING**.
- **logical_version** viene incrementata per dare priorit  alla cancellazione.

La cancellazione viaggia quindi come un normale aggiornamento ma con versione pi  alta. I peer la applicano durante il merge e non accettano record precedenti. L'eventuale rimozione fisica pu  essere rimandata fino alla propagazione completa.

3) *Rilevazione Automatica delle Scadenze (Stale Detection):* Per gestire un microservizio che termina senza deregistrarsi, il runtime esegue una **Stale Detection** periodica tramite ticker indipendente.

Per ogni servizio attivo viene calcolato il tempo trascorso dall'ultimo heartbeat:

$$\Delta t = t_{\text{current}} - t_{\text{last_heartbeat}}$$

Se $\Delta t > \text{TTL}$, il servizio viene considerato non disponibile. Il record non viene cancellato: viene trasformato in una tombstone con versione incrementata e stato **NOT_SERVING**, cos  l'informazione pu  essere replicata agli altri nodi.

G. Protocollo di replica gossip e riconciliazione dello stato

La componente **Runtime Gossip** realizza la sincronizzazione decentralizzata. Invece di bloccare l'intero cluster con un consenso sincrono, esegue quattro loop indipendenti, ciascuno dedicato a una responsabilit  specifica e regolato dal proprio intervallo temporale.

1) I Quattro Loop di Background del Runtime:

- **Bootstrap Loop (Cluster Discovery):** a intervalli regolari invia **JoinCluster** ai seed peer configurati. Il nodo si annuncia, viene inserito nella membership remota e riceve l'elenco aggiornato dei peer da aggiungere al proprio **PeerStore**.
- **Gossip Loop (Push epidemico):** seleziona pseudo-casualmente fino a k peer dal **PeerStore** e invia loro un messaggio **GossipSync**. L'informazione si diffonde in pi  direzioni senza richiedere

che ogni nodo conosca immediatamente l'intera topologia.

- **Reconcile Loop (Anti-Entropy Pull):** opera piu' lentamente del gossip e interroga un peer con `PullState`. Il parametro `since_unix` consente di chiedere gli aggiornamenti successivi all'ultimo allineamento, recuperando le informazioni perse durante errori o partizioni.
- **Peer Sweep Loop (Membership Pruning):** controlla l'eta' dell'ultima interazione con ciascun peer. Se il limite `peer_timeout_seconds` viene superato, il peer viene rimosso dalla membership attiva e non viene piu' scelto per le comunicazioni successive.

2) *Meccanismi Sinergici per la Convergenza:* La consistenza eventuale nasce dalla combinazione di tre strategie complementari:

- 1) **Propagazione proattiva (Gossip Push):** diffonde rapidamente registrazioni e tombstone verso piu' peer, fornendo il percorso principale di replica durante il funzionamento normale.
- 2) **Riallineamento reattivo (Reconcile Pull):** colma i vuoti lasciati da messaggi persi, congestione o riavvii, permettendo al nodo di recuperare lo stato dal peer interrogato.
- 3) **Ri-bootstrap sui seed peer:** mantiene un punto di rientro nella rete quando la membership locale diventa incompleta o un nodo torna operativo dopo una separazione temporanea.

VI. GESTIONE DELLA FAULT TOLERANCE

La fault tolerance del progetto combina ridondanza, monitoraggio e riallineamento. I tre registry conservano viste locali ma non dipendono da un coordinatore permanente; il guasto di un container puo' quindi ridurre la conoscenza locale senza rendere indisponibile la discovery sui peer rimasti attivi.

A. Rilevazione dei guasti

Il controllo opera su due livelli. Per i servizi applicativi, ogni heartbeat aggiorna `last_heartbeat_unix`; il ticker di `MarkStale` confronta questo valore con `heartbeat_ttl_seconds` e marca l'istanza come `NOT_SERVING` quando il TTL scade.

Per i registry, il runtime registra l'ultima interazione con ogni peer. Superato `peer_timeout_seconds`, il peer non raggiungibile viene eliminato dal `PeerStore` attivo e non viene piu' selezionato per gossip o reconcile.

B. Replica e recupero dello stato

Le registrazioni vengono diffuse con `GossipSync` verso un insieme limitato di peer. Il ciclo *reconcile* interroga periodicamente un altro nodo tramite `PullState`, recuperando le modifiche perse durante un errore o un isolamento.

`logical_version` ordina le mutazioni. Il merge applica soltanto record piu' recenti dello stato locale, rendendo idempotenti i messaggi duplicati e innocui quelli ricevuti fuori ordine.

C. Crash, recovery e graceful leave

La fault injection arresta intenzionalmente `registry-node-2`. I peer superstiti continuano a rispondere alle richieste e rimuovono il nodo fermo dopo il timeout. Al riavvio, il processo Go ricostruisce membership e record tramite seed peer, gossip e reconcile.

Il protocollo distingue un crash da uno shutdown ordinato. Con `GracefulLeave` il nodo comunica l'uscita ai peer, che lo rimuovono senza attendere il timeout, riducendo il tempo di convergenza.

D. Tombstone e continuita' del registro

La deregistrazione e lo stale detection non cancellano subito il record. Il registry crea una *tombstone*, azzera endpoint e versione applicativa, assegna `NOT_SERVING` e incrementa `logical_version`. La cancellazione viene cosi' replicata come una mutazione piu' recente e impedisce il ritorno di copie obsolete.

Il client prova gli endpoint disponibili e tratta l'errore di un singolo peer come indisponibilita' locale. Le richieste possono quindi proseguire sui nodi superstiti, realizzando una degradazione controllata.

La procedura e' verificabile con lo script operativo del progetto:

```
.\scripts\dev.ps1 trace-up
.\scripts\dev.ps1 select-service -profile random
.\scripts\dev.ps1 register
.\scripts\dev.ps1 discovery
.\scripts\dev.ps1 deregister
.\scripts\dev.ps1 down
```

La sequenza ricrea il cluster, registra un servizio, verifica la discovery, mantiene la presenza tramite heartbeat e rimuove infine il servizio e l'infrastruttura di test. Il test d'integrazione esteso replica inoltre crash, pruning dei peer, persistenza dello snapshot, resume del nodo e convergenza della deregistrazione.

VII. STRATEGIA DI TEST E VALIDAZIONE

La validazione comprende test unitari, test sui server gRPC, test di integrazione e prove Docker. I test unitari verificano regole applicative, validazione, versionamento, heartbeat e tombstone; i test gRPC coprono registrazione, discovery, deregistrazione, idempotenza e membership.

I test di integrazione avviano nodi su porte temporanee e controllano convergenza dei servizi, pruning dopo crash, persistenza e ripristino dello snapshot, riallineamento del nodo riavviato, propagazione della deregistrazione e graceful leave senza attesa del timeout.

La suite standard si esegue con:

```
go test ./...
```

Il ciclo containerizzato si riproduce con:

```
.\scripts\dev.ps1 trace-up
.\scripts\dev.ps1 select-service -profile random
.\scripts\dev.ps1 register
.\scripts\dev.ps1 discovery
.\scripts\dev.ps1 deregister
.\scripts\dev.ps1 down
```

La build delle immagini viene eseguita nella fase iniziale; i comandi CLI successivi riutilizzano l'immagine già disponibile. Per la fault injection si arresta e si riavvia **registry-node-2**, osservando risposte, pruning e recupero dello stato.

VIII. CONCLUSIONI

Il progetto realizza un registro distribuito con tre nodi cooperanti, API gRPC tipizzate e replica gossip. Heartbeat, TTL, versionamento, tombstone, pruning e reconcile coprono sia l'assenza di un servizio sia l'arresto temporaneo di un registry.

La separazione tra dominio, applicazione, adapter, storage, gossip e bootstrap rende più chiari i confini del codice. Docker Compose e **scripts/dev.ps1** forniscono inoltre un ambiente riproducibile per build, test, discovery e simulazione dei guasti.

Nel complesso, la soluzione mostra come ottenere disponibilità e consistenza eventuale senza un coordinatore centrale, mantenendo strumenti concreti per osservare e verificare il comportamento del cluster.